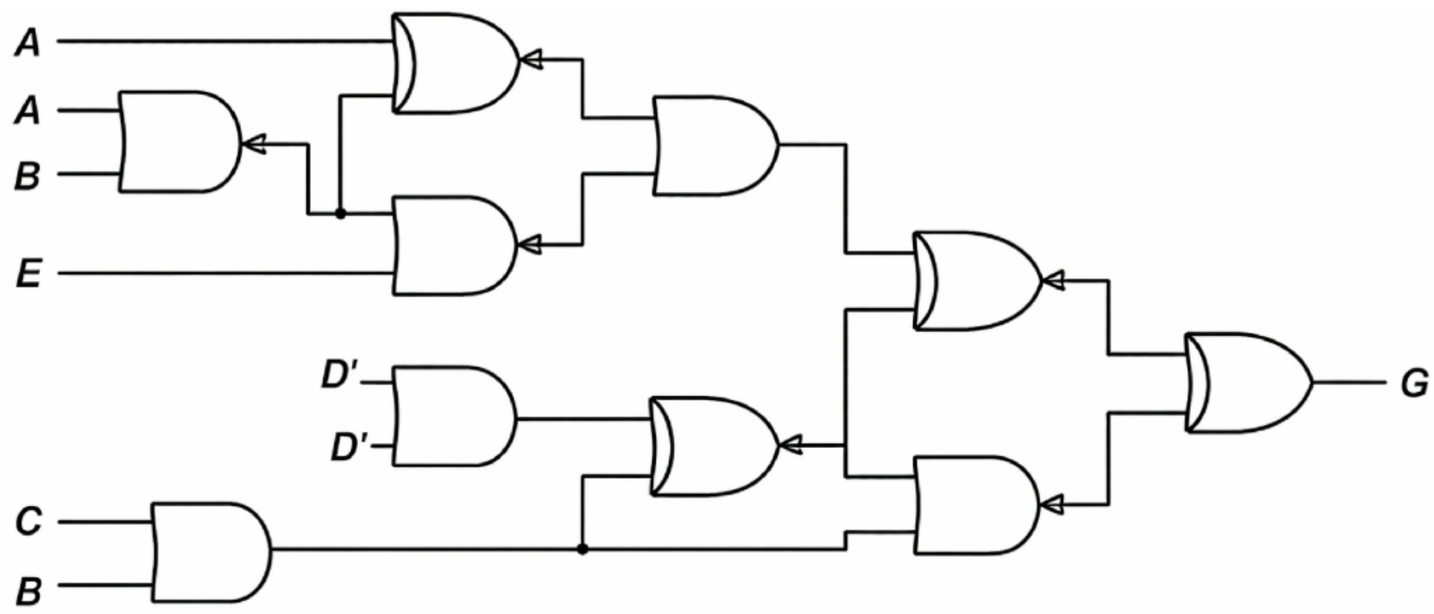
23010852 신주은 디지털논리회로 과제3(4.16목)

1. 다음 회로의 최대 지연을 구하라. 또한 입력 D'부터 출력까지의 최대 지연을 구하라.(모든 입력은 보수와 비보수 모두 가능하다고 가정함.)



2. 최대 지연 (Critical Path) 재계산

일반적으로 모든 게이트의 지연 시간을 1τ (1단위)라고 가정할 때, 가장 많은 게이트를 통과하는 경로를 찾습니다.

- **가장 긴 경로 (Critical Path):** 이미지 상단의 A 또는 B에서 시작하여 출력 G까지 가는 경로가 가장 복잡합니다.
  1. G_1 (AND: A, B 입력)
  2. G_2 (XOR: G_1 결과와 A 입력)
  3. G_3 (AND: G_2 결과와 E 입력)
  4. G_4 (OR: G_3 결과와 다른 경로 합류)
  5. G_5 (AND: 최종 출력 G 직전 게이트)

  따라서 **회로 전체의 최대 지연은 5τ (5단위)**입니다.

3. 입력 D부터 출력 G까지의 지연

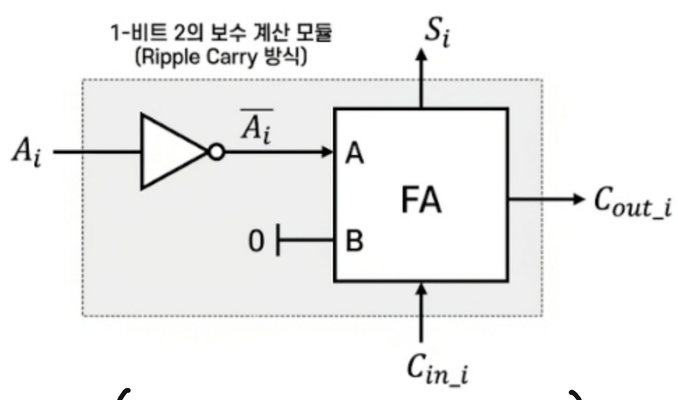
문제에서 D 입력 부분(중간)을 보면 D' 기호가 있는 XOR 게이트 라인을 따릅니다.

1. D'가 첫 번째 **XOR**를 통과 (1τ)
2. 두 번째 **XOR**를 통과 (2τ)
3. **OR** 게이트를 통과 (3τ)
4. 최종 **AND** 게이트를 통과 (4τ)

따라서 **입력 D로부터의 최대 지연은 4τ (4단위)**입니다.

3. n비트 숫자의 2의 보수를 계산하는 회로를 만들고자 한다. n개의 모듈로 ripple carry방식을 사용하여 구현하려고 한다. 다음 물음에 답하여라.

a.각 모듈은 ripple carry방식이므로 서로 같다. 그 중 한 모듈을 보여라.



(Gemini가 그린 모듈임.)

b.n비트에 대한 지연을 계산하여라.

c.이 회로의 속도를 향상시킬 수 있는 방법을 글로 설명하라.

b. n비트에 대한 지연 계산

Ripple Carry 방식의 핵심은 **캐리(Carry)가 하위 비트에서 상위 비트로 순차적으로 전달**되어야 한다는 것입니다.

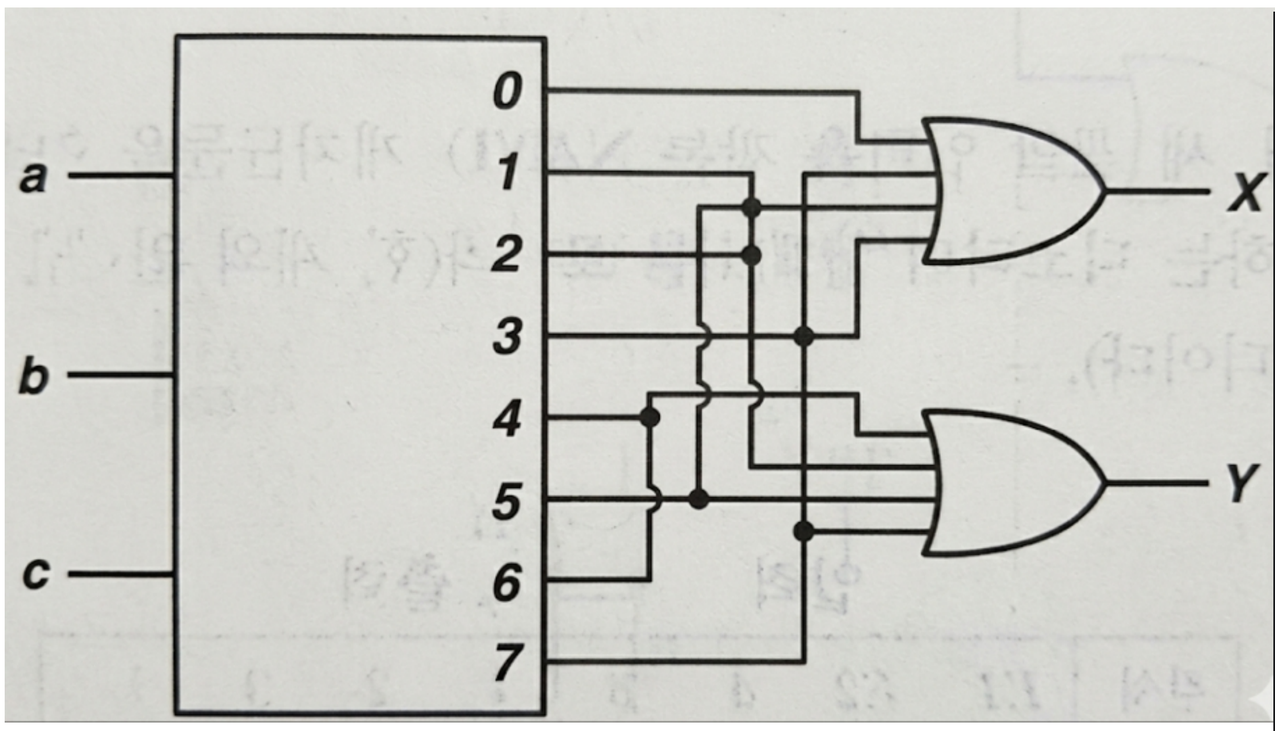
- **단일 모듈 지연 (t):** 한 모듈에서 캐리가 생성되어 다음 모듈로 전달되는 데 걸리는 시간을 t라고 합시다.
- **전체 지연:** n개의 모듈이 직렬로 연결되어 있으므로, 마지막 비트의 결과가 나올 때까지는 각 모듈의 지연이 누적됩니다.
- **결과:** $n \times t$ (즉, 비트 수 n에 **비례**하는 지연이 발생합니다.)

c. 회로의 속도를 향상시킬 수 있는 방법

Ripple Carry의 고질적인 문제는 n이 커질수록 지연이 선형적으로 늘어난다는 것입니다. 이를 해결하기 위한 방법은 다음과 같습니다.

1. **Carry Look-ahead Adder (CLA) 도입:**
   각 비트의 캐리가 발생할지 여부를 하위 비트들의 입력을 보고 **미리 계산(Look-ahead)**하는 로직을 추가합니다. 이렇게 하면 캐리가 전달되기를 기다리지 않고 한꺼번에 계산할 수 있어 지연 시간이 $O(\log n)$으로 줄어듭니다.
2. **전용 2의 보수 로직 사용:**
   단순 가산기를 쓰는 대신, "오른쪽에서 첫 번째 1이 나타날 때까지는 그대로 두고, 그 왼쪽 비트들만 반전시킨다"는 2의 보수 특성을 이용한 전용 조합 논리 회로를 설계하여 경로(Path)를 단축할 수 있습니다.

7. 다음 회로에서 입력 a,b,c로서 X와 Y에 대한 진리표를 그려라. (이 회로는 Active high 출력 디코더를 사용함.)



입력 a	입력 b	입력 c	디코더 활성 단자	출력 X	출력 Y
0	0	0	m_0	1	0
0	0	1	m_1	1	0
0	1	0	m_2	1	0
0	1	1	m_3	1	0
1	0	0	m_4	0	1
1	0	1	m_5	0	1
1	1	0	m_6	0	1
1	1	1	m_7	0	1

12. 2갈래 멀티플렉서를 사용하여 다음 함수를 구현하라.

f(x,y,z)=Σm(1,2,3,4,6)

12번. 2갈래 멀티플렉서(2-to-1 MUX)를 사용한 함수 구현

함수: $f(x, y, z) = \sum m(1, 2, 3, 4, 6)$

2-to-1 MUX를 사용하여 구현할 때, 선택 신호를 x로 잡으면 나머지 y, z에 대한 함수로 입력을 결정합니다.

x	y	z	f	x가 0일 때 (I_0)	x가 1일 때 (I_1)
0	0	0	0	$f = y + z$	
0	0	1	1		
0	1	0	1		
0	1	1	1		
1	0	0	1		$f = \bar{z}$
1	0	1	0		
1	1	0	1		
1	1	1	0		

- **구현:**
  - **선택선(S):** x
  - I_0 **입력:** y OR z (또는 $y + z$)
  - I_1 **입력:** $\bar{z}$ (NOT z)
  - 이렇게 연결하면 2-to-1 MUX 하나와 약간의 게이트로 해당 함수를 구현할 수 있습니다.